Type 1: Research Project

A Multi-Protocol Cyber-Deception Framework Combining Language-Model Response Synthesis with Blockchain-Anchored Forensic Logging

Team Number: 58

S.No	Register Number	Student Name
1	CB.SC.U4CSE23129	Sisir Reddy Nellore
2	CB.SC.U4CSE23611	Pranav Sharma Chouduru
3	CB.SC.U4CSE23127	Surya Tejeswar Reddy Nagireddy
4	CB.SC.U4CSE23111	Guru Sai Prasad Reddy Bompalli
5	CB.SC.U4CSE23644	Vishnu Sathwick Rajala

Guide: **Dr. Radhika G**

Department of CSE

Problem Statement

Low- and medium-interaction honeypots answer from a **fixed, deterministic response table** — cheap and safe, but structurally enumerable.

- **Fingerprintable by design.** Vetterl and Clayton fingerprinted low/medium-interaction honeypots at Internet scale with a *single packet*, equal-error rate 0.0183, locating 7,605 instances across nine implementations.
- **Single-surface LLM deception.** Generative honeypots that exist (ShellLM, HoneyGPT, HoneyLLM) target one Linux shell each — no shared engine spans multiple heterogeneous protocols.
- **Untrustworthy forensic logs.** A honeypot's entire purpose is to be attacked; a compromised host can have its own logs silently edited, destroying the evidentiary value of what it captured.

Objective

Build a deployable, multi-protocol deception framework that stays convincing under probing and produces a tamper-evident audit trail:

- **Cover five distinct attack surfaces** — SSH, an internal AI API, PostgreSQL, an Ethereum JSON-RPC node, and Jenkins CI/CD — behind one logging proxy.
- **Stay fast where routine, smart where novel.** Answer common traffic at static-honeypot speed; reserve LM generation for genuinely novel input.
- **Anchor forensic integrity independently of the host.** Hash and commit every record on-chain so tampering is detectable even if the honeypot itself is compromised.
- **Classify intent transparently.** Label every interaction with an auditable, rule-based classifier suitable for forensic use.
- **Evaluate honestly.** Measure latency, diversity, realism, and cost as separate axes on real payloads — and report the failures, not only the favourable numbers.

Research Gaps We Address

- **Generative deception is single-protocol.** ShellLM, HoneyGPT, HoneyLLM and the wider 2023–2025 line apply LM response generation only to an SSH/terminal shell; no prior system applies one engine across five heterogeneous protocols.
- **No system couples generation with tamper-evident logging.** Blockchain hash-anchoring is established in digital-forensics logging, but no honeypot-specific paper combines it with a generative decoy layer.
- **Realism is reported, rarely interrogated.** Prior work reports a single realism figure without separating what layer of fingerprinting it actually defends against, or admitting the timing signature a tiered design creates.

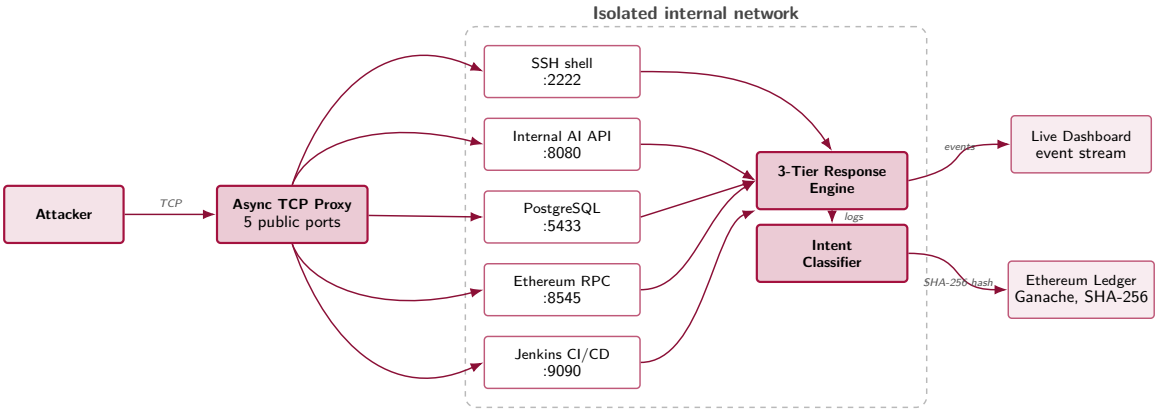
Comparison Against Cited Systems

System	Surfaces	Response mechanism	Routine latency	Log integrity
Cowrie / Kippo	1 (SSH)	Static table	Microseconds	None
ShellLM	1 (shell)	Generative	Seconds	None
HoneyLLM	1 (shell)	Generative + filter	1.88 s mean	None
HoneyGPT	1 (shell)	Generative + state manager	Seconds	None
This work	**5**	**Tiered: static, cache, generative**	**0.8 ms**	**On-chain per record**

No prior system spans more than one protocol or anchors its logs. This work is the only one in the comparison that is both multi-surface and tamper-evident. *Table IV, paper Section 10.4*

System Architecture — Overview

Only the proxy and dashboard are reachable from outside. Decoys, the response engine, the datastore, and the blockchain node sit behind it on an isolated internal network.



Paper Section 4

Five Decoy Services

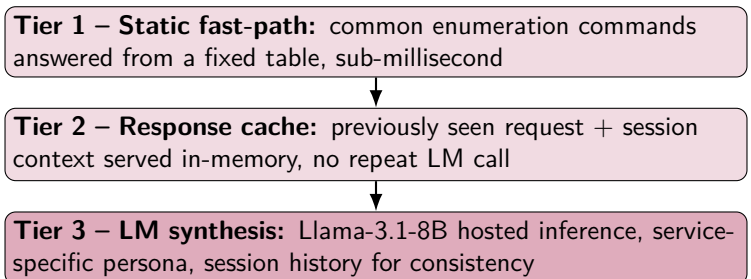
Decoy	Port / Protocol	Role
SSH shell	2222 / SSH	Real SSH server; accepts any credentials; interactive shell as a corporate Ubuntu database host; commands route through the response engine
Internal AI API	8080 / REST	Chat/predict endpoints for a fictitious internal model; target for prompt injection and jailbreak attempts
PostgreSQL endpoint	5433 / SQL	Accepts SQL queries; returns plausible tabular results or realistic error text, baiting SQL injection
Ethereum JSON-RPC	8545 / RPC	Responds to the RPC methods attackers invoke to enumerate and drain wallets
Jenkins CI/CD	9090 / HTTP	Build- and script-oriented endpoints representing the documented Jenkins attack surface

Paper Section

4.2

Three-Tier Response Path

Generating every response with the LM would be slow, costly, and, by its uniform multi-second latency, itself detectable.



Intent classification: rule-based, 8 categories (SQLi, prompt injection, crypto theft, cryptomining, reverse shell/RCE, privilege escalation, supply-chain/CI-CD abuse, reconnaissance) — auditable and explainable, chosen over a learned classifier for forensic use. *Paper Section 4.3–4.4*

Blockchain-Anchored Logging

- Every record is intent-classified, serialised deterministically, and **SHA-256 hashed**.
- The hash is anchored as a transaction on a local Ethereum chain (Ganache); the full record stays in the datastore, only the digest goes on-chain.
- Any post-hoc edit to a stored record changes its hash, which no longer matches the on-chain digest — **detectable without trusting the honeypot host**.
- Integrity, not veracity: the anchor proves a record was not altered after capture. It does not prove the (synthesised) response describes a real system state.

Integrity Test Result

An untampered record's recomputed SHA-256 digest matched the anchored value exactly. After altering a single field, the recomputed digest diverged from the anchored value — confirming tamper-evidence.

Paper Section 4.5, 6.5, 8.5

Evaluation Methodology

Evaluated the LM-backed dynamic path against a static-response baseline on **2,500 real, publicly documented** attack payloads (SecLists, PayloadsAllTheThings) across all five services.

- **Four independent axes, kept separate:**
  - Latency / sequential throughput and concurrent throughput (10 simultaneous sessions, fresh nonce per request to force a genuine model call)
  - Response diversity — uniqueness ratio per service
  - LM-judge realism — independent Llama-3.1-70B judge, 1–5 rubric, 30-payload sample
  - Blockchain integrity — direct tamper test
- Payload split: 1,862 SSH, 474 SQL, 136 API, 18 Web3, 10 CI/CD — capped at the real payloads genuinely available per surface, no padding.
- Corpus is **single-request payloads evaluated independently** — a stated limitation, not a multi-stage campaign simulation.

Paper Section 5

Corpus Coverage by Adversary Behaviour

Partition	n	Adversary behaviour represented	ATT&CK
SSH	1,862	Command injection, shell enumeration, reverse-shell staging, credential access, cryptomining install	T1059.004, T1082, T1071, T1552
SQL	474	SQL injection, schema enumeration, authentication bypass, data exfiltration via query	T1190, T1213
API	136	Prompt injection, jailbreak, XXE, SSRF against an internal model-serving endpoint	T1190, T1552.001
Web3	18	Wallet enumeration, unauthorised account unlock and transfer against exposed JSON-RPC	T1190, T1657
CI/CD	10	Build-system abuse, script-console RCE, credential harvesting from a CI server	T1195, T1059

Payloads are real strings observed or catalogued in the wild — the corpus covers the *content* of a broad range of contemporary attacks. It does **not** cover multi-stage campaigns: every payload is evaluated as a single, independent request. *Table I, paper Section 5.2*

Latency and Throughput

Metric	Static fast-path	Dynamic (LM) path
Mean latency	0.8 ms	2.33 s
Median latency	0.8 ms	1.73 s
Std. deviation	0.3 ms	1.40 s
Sequential throughput	≈1,250 req/s	0.43 req/s

The two paths are roughly **four orders of magnitude** apart — the explicit justification for tiering: routing routine traffic to the fast-path is what makes an LM-backed honeypot viable at all. *Paper Section 6.1*

Concurrency Scaling: Where the Ceiling Really Is

Ten simultaneous sessions sustain **1.61 req/s** at 2.19 s mean per-request latency (6.21 s wall time for the burst).

- Ideal linear scaling at concurrency 10 would give 4.29 req/s. Measured 1.61 req/s $\Rightarrow$ speedup 3.76$\times$, parallel efficiency 0.376.
- Little's Law: requests in flight $= 1.61 \times 2.19 \approx 3.53$, not 10 — the system saturates at an **effective parallelism of $\approx$3.5**.
- Per-request latency under load (2.19 s) is *lower* than sequential latency (2.33 s) — if our framework queued requests, load would inflate latency. It does not.

Diagnosis

The bottleneck is upstream admission control at the **hosted inference provider**, not a queue in the proxy, decoys, or response engine. Raising provider concurrency, not re-engineering the framework, is the fix.

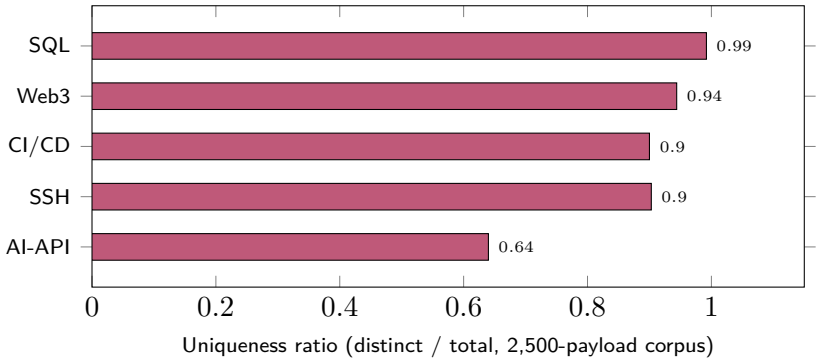
Paper Section 6.2

Connection Pooling: Decoy-Layer Latency, All Five Decoys

Decoy	Endpoint	Before (mean)	After (mean)	Change
oracle-ai	`GET /api/models`	6.15 ms (σ 3.89)	4.28 ms (σ 0.70)	-30.4%
vault-sql	`GET /health`	0.69 ms (σ 1.94)	0.41 ms (σ 0.44)	-41.4%
ledger-trap	`POST /` (`eth_accounts`)	5.96 ms (σ 2.43)	4.20 ms (σ 0.75)	-29.5%
forge-ci	`GET /api/json`	6.95 ms (σ 3.13)	6.71 ms (σ 3.02)	-3.4%
shadow-shell	SSH `whoami`	7.91 ms (σ 9.94)	4.64 ms (σ 6.55)	-41.3%

A single pooled HTTP client, reused for the process lifetime, cut latency by **30–41%** on four of five decoys and tightened variance by half to two-thirds. `forge-ci` barely moves — it routes through the response engine regardless of decoy-layer pooling, so its floor is the cache lookup. *Table V, paper Section 11.1*

Response Diversity by Decoy Service



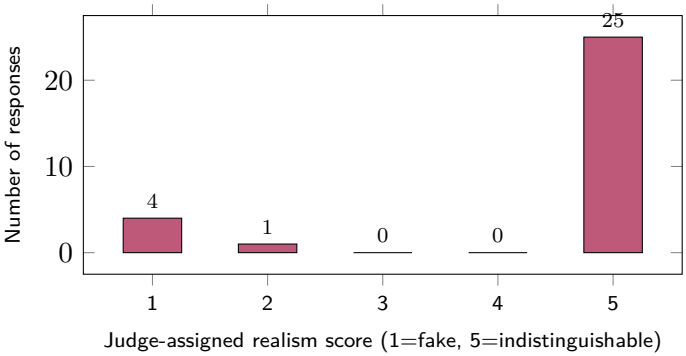
AI-API is the clear outlier — a meaningful minority of responses to structurally similar payloads (XXE, SSRF probes) converge on near-identical generic error text. *Table II, paper Section 6.3*

Per-Service Results: Diversity and Realism Agree

Service	Payloads	Distinct outputs	Uniqueness ratio	Judged realism
SQL	474	470	0.992	High
Web3	18	17	0.944	High
SSH	1,862	1,681	0.903	High, small templated minority
CI/CD	10	9	0.900	High
AI-API	**136**	**87**	**0.640**	**Lowest – most low ratings**

Mean LM-judge realism: **4.37/5** ($\sigma = 1.45$, $n = 30$). Distribution strongly bimodal: 25 rated 5, one rated 2, four rated 1, none rated 3 or 4 — usually in character, occasionally a complete lapse. *Table II, paper Section 6.4*

Realism Score Distribution ($n = 30$)



Low ratings concentrate on, but are not exclusive to, the AI-API decoy: generic error text that ignores the specific XML/SSRF probe, plus a few templated SSH fallbacks. *Paper Section 6.4*

Blockchain Integrity: Direct Tamper Test

Independently of realism and diversity, we verify the tamper-evidence claim directly rather than assuming it holds.

- Fetch a stored record, recompute its SHA-256 digest, and confirm it matches the value already anchored on-chain.
- Modify one field of that record, re-hash, and confirm the digest now **diverges** from the anchored value.
- Repeat for a record that was never tampered with, to rule out a false-positive detector.

Result

For an untampered record the recomputed digest matches exactly. After a single-field edit, the recomputed digest diverges from the anchored value every time — tampering is detected without trusting the honeypot host that produced the record.

An attacker who compromises the honeypot and edits a stored record **cannot** make the edited record's hash match what was already committed on-chain. This is integrity, not veracity: the anchor says the record is unaltered, not that its (synthesised) content is true. *Paper Section 6.5*

Resistance to Fingerprinting: Four Layers, One Metric

Response diversity is the metric most generative-honeypot papers report. It addresses **one of four** fingerprinting layers.

Layer	Attack	Our defence	Evaluated?
1. Transport	Protocol-library deviation (Vetterl & Clayton)	None specific	No
2. Timing	Tier-boundary latency ratio	None currently	Quantified; not mitigated
3. Content	Response-set enumeration	Generative synthesis, 0.64–0.99 uniqueness	**Yes**
4. Semantic	Cross-command consistency	Session-history prompting	No

High

response diversity buys resistance to **one specific, common** class of automated fingerprinting. Two other layers remain open. *Table III, paper Section 7*

Finding: The Tiered Design Is Its Own Timing Tell

The static fast-path answers in **0.8 ms** ($\sigma = 0.3$ ms). The model path answers in **2.33 s** ($\sigma = 1.40$ s). The distributions do not overlap — separated by ≈3.5 orders of magnitude.

Finding 1 — a characterised timing side-channel

Sending two commands, one common (`whoami`) and one arbitrary, and comparing response times yields a ratio near **3,000** on our system. No real service exhibits that ratio between two commands of comparable output size. **A single pair of probes is sufficient to detect the tiered architecture with high confidence.**

This is a direct consequence of the design choice that makes the system economically viable. The tiering that keeps routine traffic cheap is also the clearest fingerprinting surface it presents. *Paper Section 7.2*

Security Risks the Generative Layer Introduces

- **Prompt injection against the decoy.** Attacker input reaches the model's context directly; a successful override converts suspicion into confirmation. The AI-API decoy is the acute case — it *advertises* itself as model-serving.
- **Cost exhaustion via forced cache misses.** A unique nonce per request defeats the cache and forces a model call every time — our own benchmark harness does exactly this. At saturated capacity ($\approx$1.6 req/s) an adversary sustains $\approx$**138,000 model calls/day**: a denial-of-wallet attack.
- **Injection into the operator console.** The dashboard renders attacker-influenced, model-generated text; unescaped output is stored XSS against the defender's own console.
- **Integrity is not veracity.** The blockchain anchor proves a record was not altered — it says nothing about whether the (synthesised) content is true.

Paper Section 8

Cost Analysis

$$\text{Cost per 1,000 interactions} = 1{,}000 \times (1 - h) \times c$$

where h = fraction absorbed by the static path + cache, c = price per model call.

- Mean dynamic response ≈19 output tokens; per-call cost is a small fraction of a US cent at commodity hosted pricing.
- **We withdraw the original manuscript's unmeasured 80% fast-path hit-rate estimate.** Our evaluation forced model calls on 2,500 distinct payloads by design — it bounds usage from above, it does not measure a representative h.
- The benign case is *not* the binding constraint: Section 8 shows an adversary can force $h \to 0$, generating ≈138,000 calls/day at saturated throughput.

Necessary architectural feature, not yet built

A hard cost ceiling with a defined degradation mode (fall back to static responses under attack) is required for any exposed deployment. Its absence is a current limitation.

Paper Section 9

Comparison With Prior Systems

System	Routine latency	Novel-request latency	Log integrity
Cowrie / Kippo	Microseconds	n/a (static)	None
ShelLM (human study)	Seconds	Seconds	None
HoneyLLM	1.88 s mean	1.88 s mean	None
This work	**0.8 ms**	**2.33 s mean**	**On-chain per record**

Not a controlled head-to-head — each figure reproduced as published, under different hardware/models. What holds: our fast-path matches static-honeypot speed while sitting in the same latency band as published mid-size generative systems on the novel tail, across five surfaces instead of one, with integrity none of them offer. *Table IV, paper Section 10*

Engineering Gaps Found and Fixed

Gap	**Fix and effect**
Naive design routed every command to the LM (≈4.0 s mean)	Introduced 3-tier path + pooled HTTP client + shorter generation cap → ≈1.35 s mean
Decoy-layer requests opened a fresh connection per call	Connection pooling: −30% to −41% latency on 4/5 decoys, variance cut by half to two-thirds
Benchmark harness reused payloads → measured **cache**, not the model	Fresh nonce per concurrent request → corrected throughput from "hundreds of req/s" to the real 1.61 req/s
Intent classifier labelled ≈75% of traffic `unknown`	Expanded rule categories → unknown fraction reduced to effectively zero on a 1,000-sample

Kept deliberately in the paper: a benchmark silently measuring its own cache is a common, easily missed mistake — and the same nonce trick is also an attack (Section 8). *Paper Section 11*

Limitations & Validation Roadmap

- **Timing side-channel (most serious):** architectural, cheap to exploit, not yet mitigated.
- **AI-API decoy** trails the other four on both diversity and realism — prompt under-constrained relative to a shell or SQL engine's rigid output format.
- **Transport-layer fidelity unaudited** on 4 of 5 decoys — the exact layer the strongest published fingerprinting attack operates on.
- Realism is measured by an **automated LM-judge proxy**, $n = 30$ — not a human-subject study.
- Corpus is single-request; **no multi-stage, cross-session consistency** evaluation yet.
- Fast-path hit rate h under live traffic is unmeasured; the original 80% estimate is withdrawn.

Next, in order of value: persona-escape rate (cheapest), human-subject realism study, multi-stage attack chains, timing-jitter mitigation curve, load sweep to $c{=}100$, live-traffic h measurement. *Paper Sections 12–13*

References

[1] L. Spitzner, *Honeypots: Tracking Hackers*, Addison-Wesley, 2002.

[2] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," 2015. `github.com/cowrie/cowrie`

[3] The Honeynet Project, "Dionaea: A Malware-Capturing Honeypot," 2016.

[4] A. Vetterl and R. Clayton, "Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale," *Proc. 12th USENIX WOOT*, 2018.

[5] F. McKee and D. Noever, "Chatbots in a Honeypot World," arXiv:2301.03771, 2023.

[6] M. Sladić, V. Valeros, C. Catania, and S. Garcia, "LLM in the Shell: Generative Honeypots," *Proc. IEEE EuroS&PW*, 2024.

[7] Z. Wang et al., "HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model," arXiv:2406.01882, 2024.

[8] W. Fan, Z. Yang, Y. Liu, L. Qin, and J. Liu, "HoneyLLM: A Large Language Model-Powered Medium-Interaction Honeypot," *Proc. ICICS*, 2024.

[9] R. A. Bridges et al., "SoK: Honeypots & LLMs, More Than the Sum of Their Parts?," arXiv:2510.25939, 2025.

[10] M. Boffa et al., "LogPrécis: Unleashing Language Models for Automated Malicious Log Analysis," *Computers & Security*, vol. 141, 2024.

[11] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[12] A. Sutton and R. Samavi, "Blockchain Enabled Privacy Audit Logs," *Proc. ISWC*, 2017.

[13] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," *NeurIPS*, 2023.

[14] MITRE, "MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge," 2015.

[15] D. Miessler and J. Haddix, "SecLists: A Collection of Security Testing Lists," 2012.

Thank You